

DESIGN AND DEVELOPMENT OF TOOT – A MODERN MERN-BASED MUSIC STREAMING WEB APP



Prepared by: Ichchuk Adhikari
Coventry ID : 14188363
Batch: 34a

Submitted to: Albert Maharjan

Table of Contents

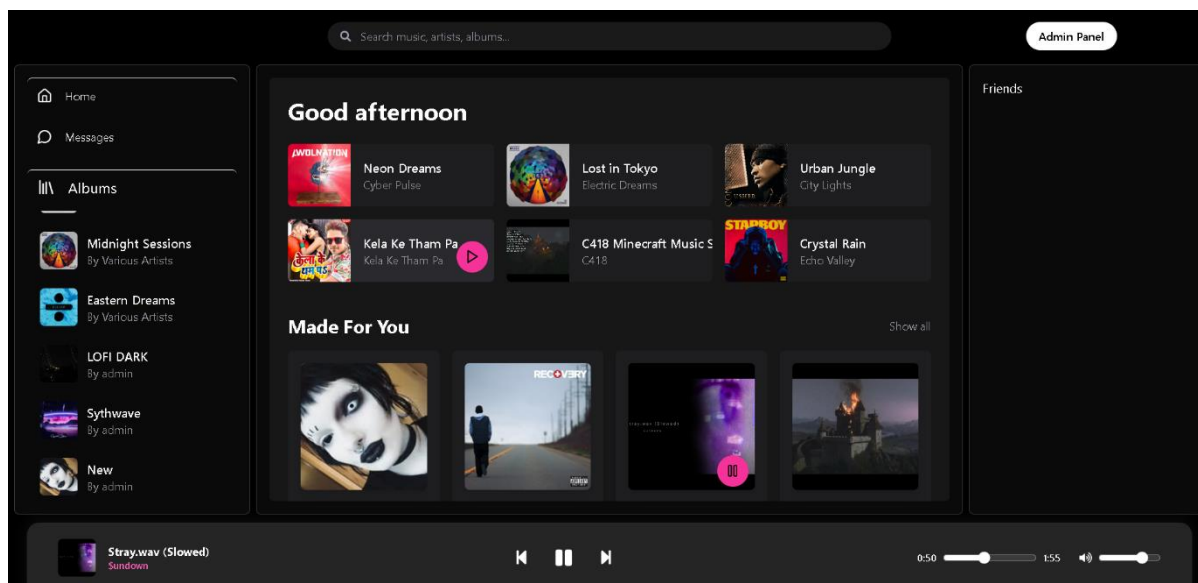
Table of Contents	2
Table of Figure	4
Introduction.....	4
Technology Stack.....	6
Backend:	7
Frontend:	8
Other tools:.....	8
REST Api Development	9
Frontend (React SPA)	9
Design pattern and Architecture.....	10
Challenges faced and Lessons Learned	11
1. Authentication and Security:.....	11
2. File Upload Management:.....	11
3. Frontend State Management	11
4. Backend Architecture.....	11
5. Testing and Error Handling.....	11
Skills Gained.....	12
Conclusion and Future Work	13
References.....	14
Appendix.....	15
1. GitHub Repositories:	15
2. Video Screencast:.....	15
3. Working Screenshots:	15
4. Test Case Logs:	24
ER Diagram	26

Table of Figure

Figure 1: Toot Music Streaming App.....	5
Figure 2: Technology Stack	6
Figure 3: Backend tools	7
Figure 4: Frontend Tools.....	8
Figure 5: MVC Architecture	10
Figure 6: Landing Page - Get Started	15
Figure 7: Landing Page - Why Choose Toot?	16
Figure 8: Landing Page - Artists	16
Figure 9: Landing Page - Preview	17
Figure 10: Landing Page - Access now.....	17
Figure 11: Login Page.....	18
Figure 12: Register Page.....	18
Figure 13: Dashboard - Main Page	19
Figure 14: Dashboard - Trending.....	19
Figure 15: : Dashboard - Album Page (1).....	20
Figure 16: Dashboard - ALbum Page (2).....	20
Figure 17: Dashboard - Search Suggestion.....	21
Figure 18: Admin Page - Songs Page	21
Figure 19: Admin Page - Add Song	22
Figure 20: Admin Page - Album Page	22
Figure 21: Admin Page - Add Album	23
Figure 22: Admin Page - Users Page	23
Figure 23: Frontend Tests	24
Figure 24: Backend Tests - 1.....	24
Figure 25: Backend Tests - 2.....	25
Figure 26: Backend Tests - 3.....	25
Figure 27: Backend Tests - 4.....	26
Figure 28: ER Diagram.....	26

Introduction

Figure 1: Toot Music Streaming App

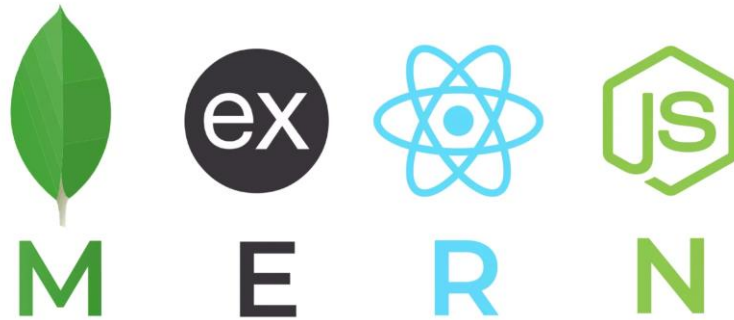


Toot is a modern, full-stack web app for streaming music that aims to give users a smooth and personalised audio experience. A clean, easy-to-use interface lets users look through trending songs, curated playlists, and albums. Toot changes the way people listen to music online, whether they're finding new artists or enjoying old favourites. It has features like real-time playback, personalised recommendations, search, and easy navigation.

The MERN stack is used to build the app, which makes it fast, responsive, and able to grow. Toot also has secure login, role-based access (like admin controls for managing content), and dynamic parts that make it easier for users to interact with each other. Toot gives you a full and immersive music streaming experience on all your devices thanks to its beautiful design and powerful features.

Technology Stack

Figure 2: Technology Stack



Toot was built using the MERN technology stack, which uses Express and MongoDB for database administration. React and server-side logic are handled by js.Node.js is utilised as the backend runtime environment, and JavaScript is utilised to produce a dynamic and responsive user interface. The application maintains a single JavaScript environment to facilitate effective client-server communication. Cloud storage services are integrated to efficiently manage audio and image files, JWT is used for secure authentication, and tools like Mongoose are used for data modelling.

Backend:*Figure 3: Backend tools*

Toot's backend was constructed with Node.js and Express.js, which are scalable and utilised by sites such as Netflix. While bcrypt protected passwords (Palmskog, 2018), JWT managed authentication (Biometrics, 2015). Multer oversaw file uploads, including audio and image files, within Express's middleware-based system, and CORS allowed secure frontend-backend communication.

Frontend:*Figure 4: Frontend Tools*

The frontend was created using React.js, which is used by Airbnb and Facebook to create responsive user interfaces (Koenig, 2021). Vite accelerated development ((n.d.), 2022). Tailwind CSS (Showcase - Build Anything, n.d.) was used for styling, along with lucide-react for icons, react-router-dom for routing, and Axios for API communication, imitating usage in Trello-like apps.

Other tools:

Additional tools include MongoDB, which is used in apps like Uber and provides flexible document-based data storage (MongoDb, n.d.). In contemporary JavaScript projects, Vitest was selected for quick unit testing, particularly with Vite (Vladimir, n.d.). A common practice in Node.js environments, Nodemon's automated server restarts on file changes improve development productivity. Across the Toot platform, these tools facilitated effective development, testing, and data handling.

REST Api Development

Key operations for songs, albums, and playlists were handled by RESTful APIs created with Express.js. In order to facilitate the creation, retrieval, updating, and deletion of data, endpoints such as `/song/createSong`, `/song/getFeaturedSong`, `/album/createAlbum`, and `/playlist/updatePlaylist/:id` were developed. By facilitating seamless communication between the frontend and backend, these routes allowed for dynamic access to the music library and user content management.

JWT was used to manage user roles and secure sensitive routes. In order to detect validation errors, unauthorised access, and server problems and guarantee consistent, informative responses, extensive error handling middleware was added. This design encouraged maintainability, security, and scalability throughout the API layer of the platform.

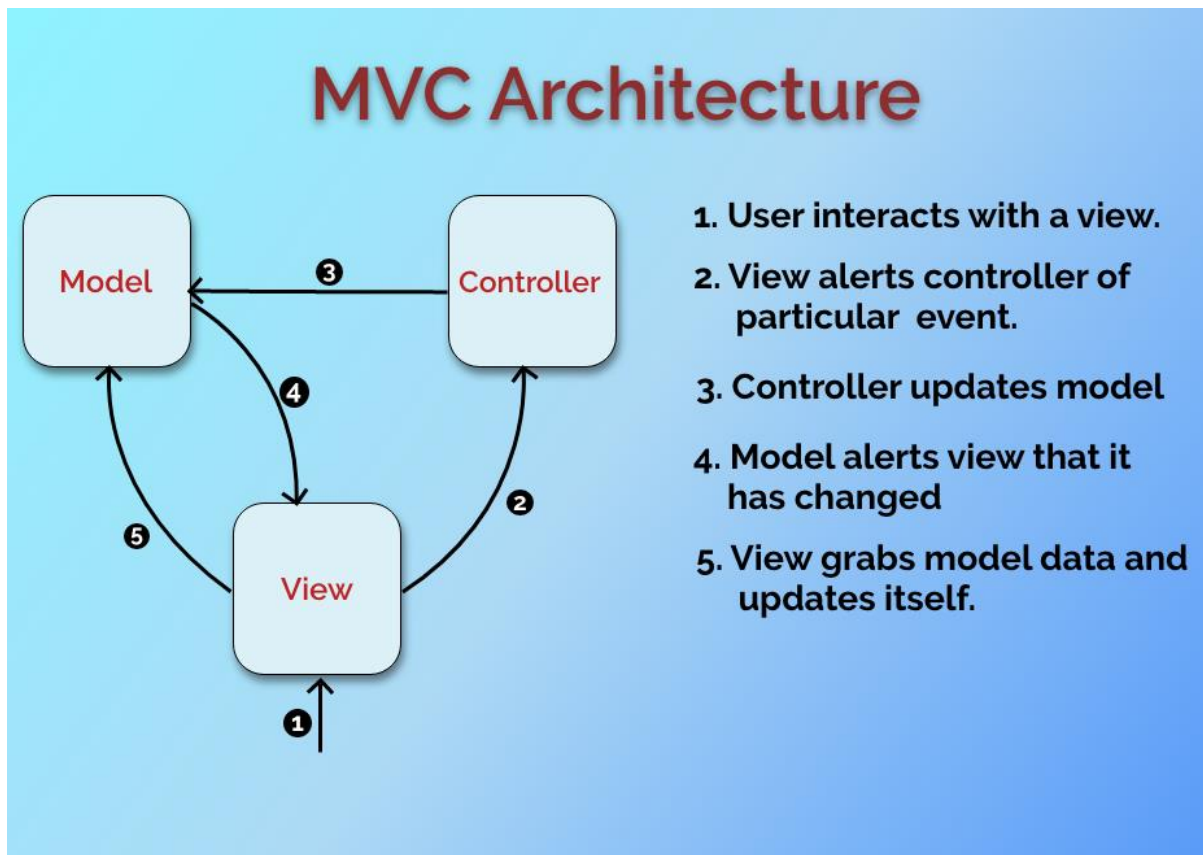
Frontend (React SPA)

To provide a smooth and interesting user experience, Toot's frontend was created as a React Single Page Application (SPA). Users can navigate through an artist's discography with ease thanks to the Album page, which offers rich information about albums, including track listings, release dates, and artist details. Comprehensive playback controls, and the ability to add tracks to playlists are all available on the Song page. Users can create, edit, and manage their playlists on the Playlist page, resulting in a more individualized music experience. React-router-dom manages the seamless navigation between these pages, guaranteeing quick transitions without requiring a reload.

Discovery and engagement features include a robust Search bar that supports real-time filtering across songs, albums, and artists, helping users find content quickly and intuitively. Based on users' listening preferences and habits, the Made For You section creates customized playlists and song recommendations, increasing user engagement with relevant content. Users are encouraged to check out the newest hits by the Trending Songs section, which showcases well-liked songs that are trending throughout the platform. To maintain frontend content in sync with the backend for a dynamic, real-time experience, the application uses Tailwind CSS for clean, responsive styling, Lucide-React for consistent iconography, and Axios for efficient API calls.

Design pattern and Architecture

Figure 5: MVC Architecture



The Toot platform encourages code reuse and uniform data management by utilising a client-server architecture to offer a single backend for web and mobile applications. Key entities like Song, Album, User, and Playlist are modelled independently on the backend using an MVC (Model-View-Controller) architecture. While routers specify RESTful API endpoints, controllers manage the main business logic and communicate with these models. Because the frontend only uses these clearly defined API routes to access data and functionality, this structure makes it easy to separate concerns and makes the system scalable and maintainable.

The frontend uses a React-built component-based architecture, which separates the user interface into independent, reusable components. React hooks ensure dynamic and responsive user experiences by managing component state and lifecycle events. To increase modularity and separation of concerns, services are put in place to manage all API communications through Axios, abstracting data fetching and updates from UI components. This architecture makes it simpler to test, scale, and collaborate, which makes it possible for the Toot platform to develop effectively as new features and enhancements are added.

Challenges faced and Lessons Learned

Within completion of this project following challenges were faced and lessons learned:

1. Authentication and Security:

- Initially, role-based access was absent; to secure authentication and safeguard user data, JWT middleware and bcrypt hashing were used.

2. File Upload Management:

- In order to efficiently validate and store audio and image files, Multer middleware was introduced because handling file uploads was challenging.

3. Frontend State Management

- It was difficult to manage asynchronous state; context and React hooks were utilised to seamlessly synchronise backend and user interface data.

4. Backend Architecture

- Creating a clear MVC structure enhanced the scalability, maintainability, and organisation of code across controllers, routes, and models.

5. Testing and Error Handling

- Unit tests and middleware were added for improved reliability and feedback because instability was caused by a lack of testing and error handling.

Skills Gained

A variety of technical skills were gained during Toot's development, such as full-stack JavaScript development using Node.js, Express, and React. Key technical accomplishments were mastery of creating safe RESTful APIs, putting JWT authentication into practice, using React hooks to manage state, and using Multer to handle file uploads. The application's overall quality and maintainability were also enhanced by proficiency with responsive design using Tailwind CSS and thorough test writing.

Important soft skills like project management, effective communication, and incorporating user feedback to refine features were developed in tandem with technical growth. Task and schedule management enhanced productivity and organisation, and cooperative problem-solving and flexibility made sure that difficulties were resolved effectively. The technical and interpersonal skills necessary for successful software development were fostered by these experiences, which improved the capacity to work both independently and collaboratively.

Conclusion and Future Work

Toot's development showed how to leverage full-stack JavaScript technologies to create a secure, scalable, and responsive music streaming platform. Authentication, testing, and a visually appealing frontend experience were all combined with necessary backend services. The project improved technical and teamwork software engineering skills and provided a solid basis for understanding real-world application development.

One significant improvement in the future will be the incorporation of Puppeteer for the purpose of extracting audio from public YouTube reels and Instagram posts. With the help of this automation, the system will be able to automatically upload reel audio to the database, classify it by genre or theme, and convert it into song files. In line with changing user interests, it will minimise manual labour and grow the music library with little assistance.

Increasing the adaptability and immersion of the user experience is another exciting future direction. The app can dynamically alter its animation and theme while recommending music that fits the mood by identifying environmental factors like the time of day (morning/night) or the weather (rainy/hot/cold). By making the listening experience feel natural, poignant, and sensitive to real-life situations, this context-aware personalisation seeks to increase user engagement.

References

- (n.d.), V. (2022). *Vite*. (n.d.). Retrieved from <https://vite.dev/>
- Biometrics, S. K. (2015). *Secure key agreement using pure biometrics*. Retrieved from <https://ieeexplore.ieee.org/document/7346828>
- IEEE. (2015). *A Horizontally-Scalable Multiprocessing Platform Based on Node.js*. Retrieved from <https://ieeexplore.ieee.org/abstract/document/7345634>
- IEEE, A. H.-S. (n.d.). *A Horizontally-Scalable Multiprocessing Platform Based on Node.js*. Retrieved from <https://ieeexplore.ieee.org/abstract/document/7345634>
- Koenig, J. R. (2021). *Adaptive restarts for stochastic synthesis*. Retrieved from <https://doi.org/10.1145/3453483.3454071>
- MongoDb, U. C. (n.d.). *Use Cases*. Retrieved from <https://www.mongodb.com/solutions/use-cases>
- Palmskog, A. &. (2018). *ACM Transactions on Modeling and Computer Simulation*. Retrieved from <https://doi.org/10.1145/3158668>
- Showcase - Build Anything, n. (n.d.). *Showcase - Build anything*. Retrieved from <https://tailwindcss.com/showcase>
- Vladimir, A. F. (n.d.). *Vitest. Next Generation Testing Framework*. Retrieved from <https://vitest.dev/>

Appendix

1. GitHub Repositories:

Frontend: <https://github.com/alchchuk/Web-API-Front>

Backend: <https://github.com/alchchuk/Web-API-Back/>

2. Video Screencast:

YouTube: <https://www.youtube.com/watch?v=60TsyvVdtOQ>

3. Working Screenshots:

Figure 6: Landing Page - Get Started

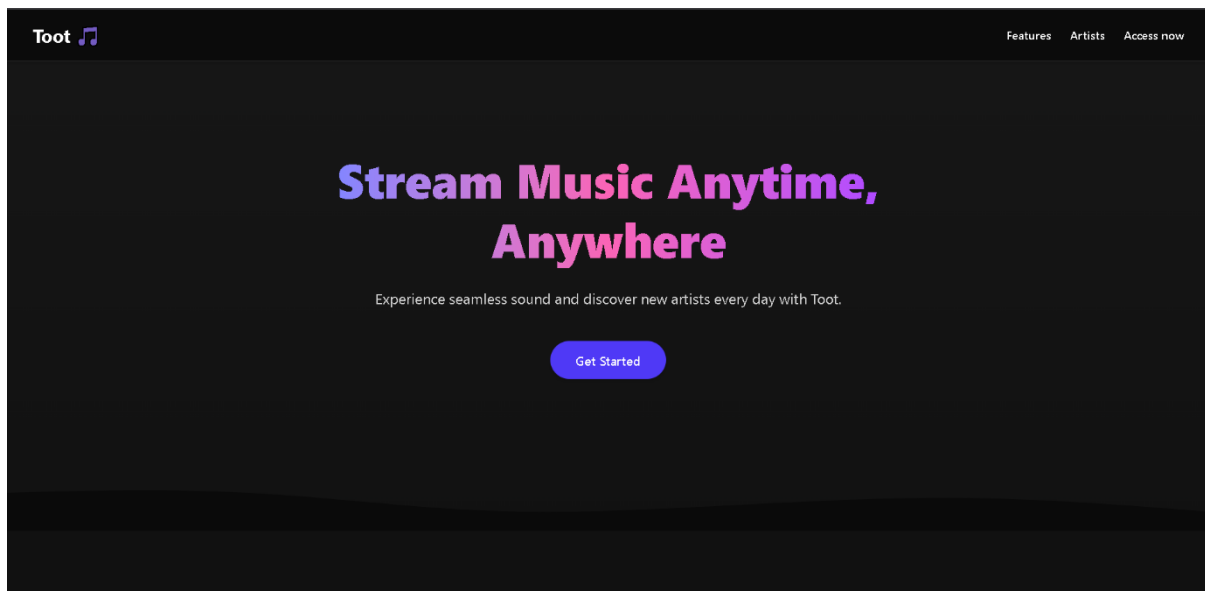


Figure 7: Landing Page - Why Choose Toot?

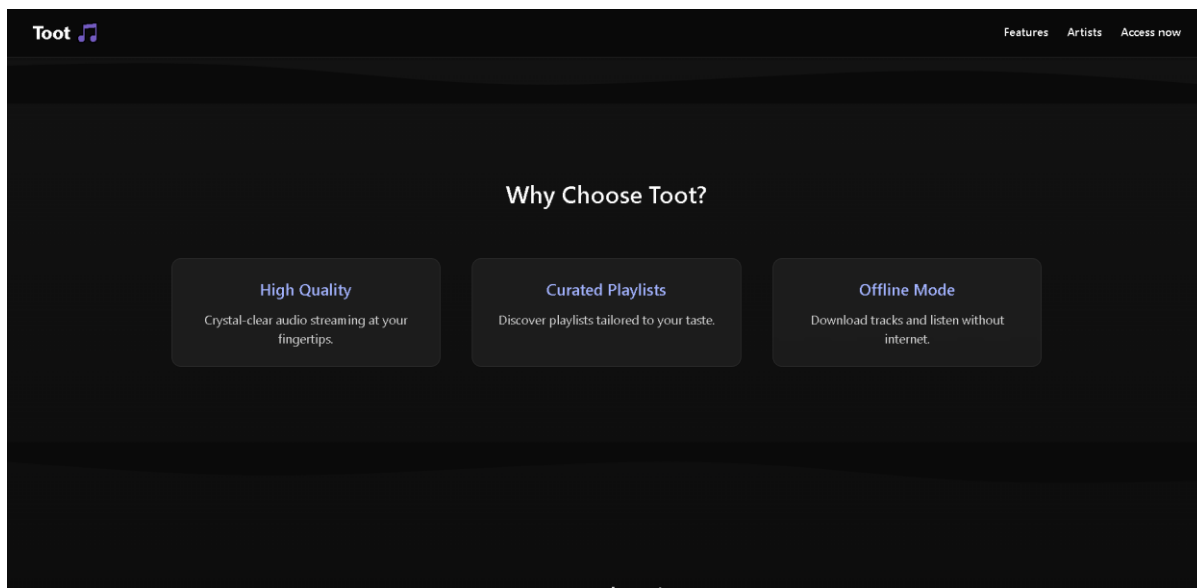


Figure 8: Landing Page - Artists

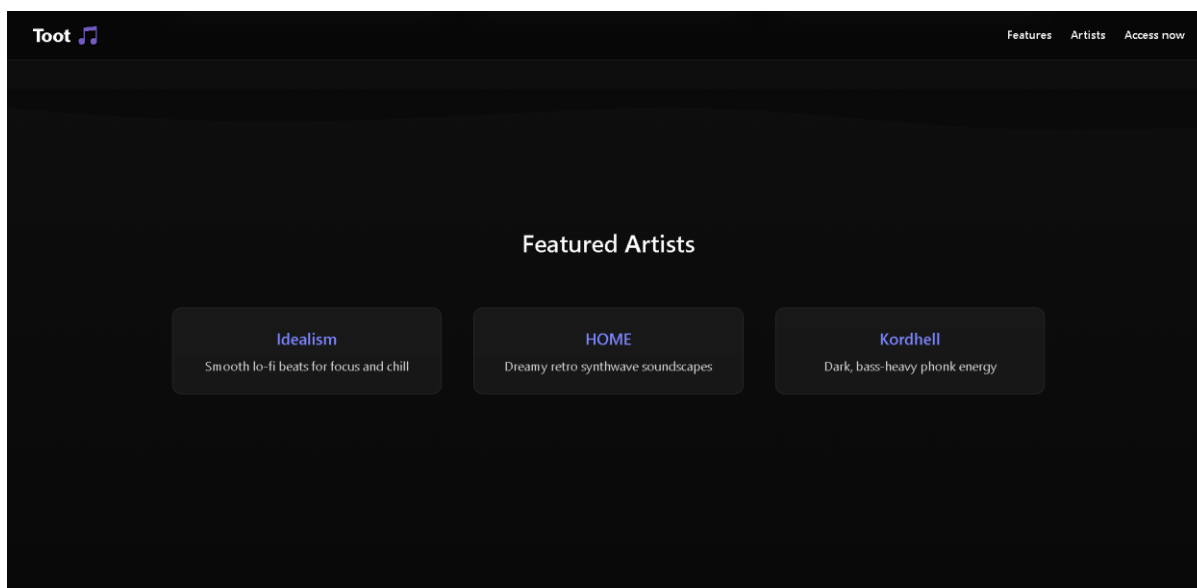


Figure 9: Landing Page - Preview

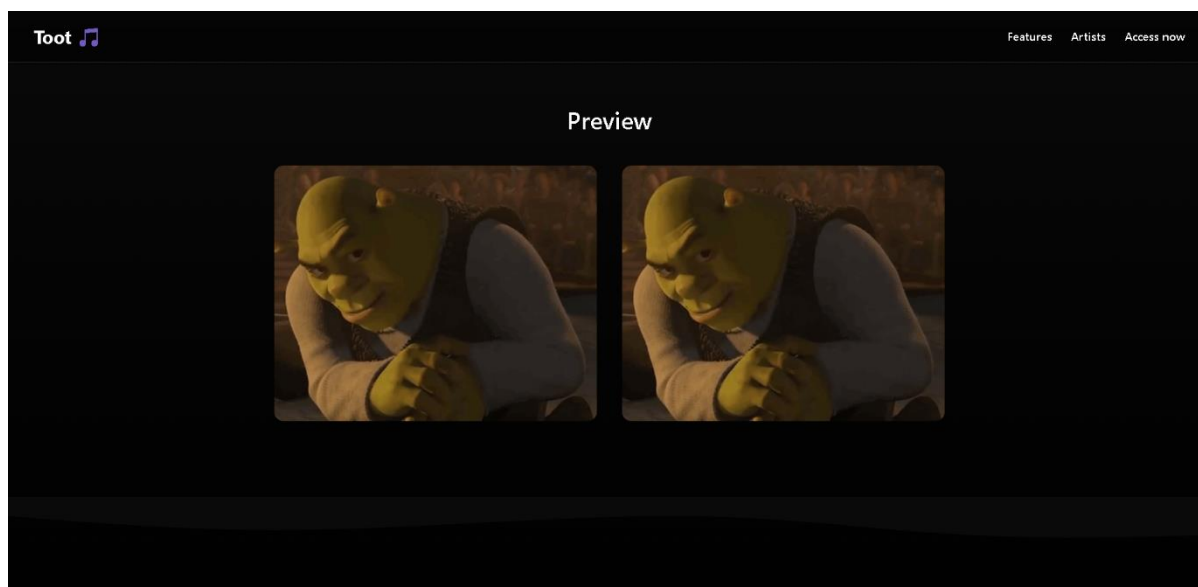


Figure 10: Landing Page - Access now

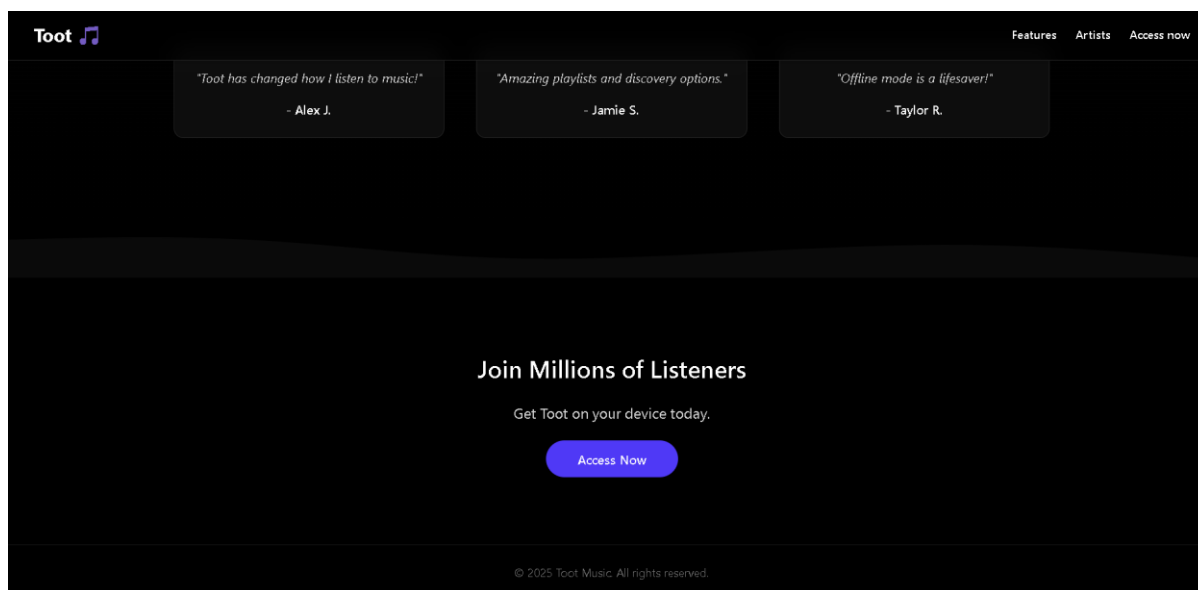
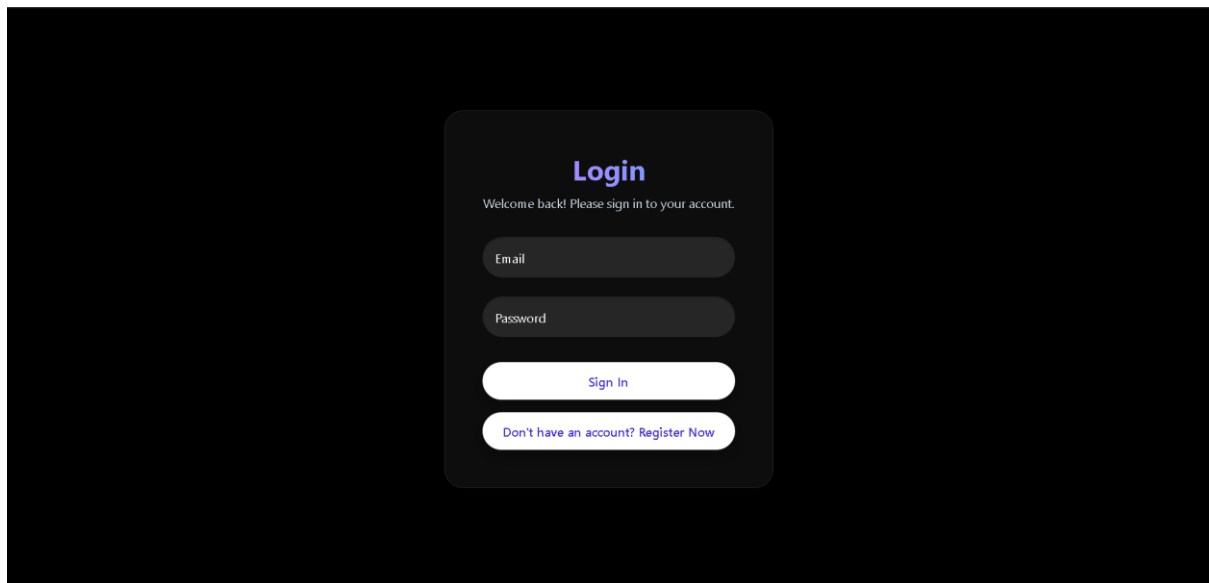
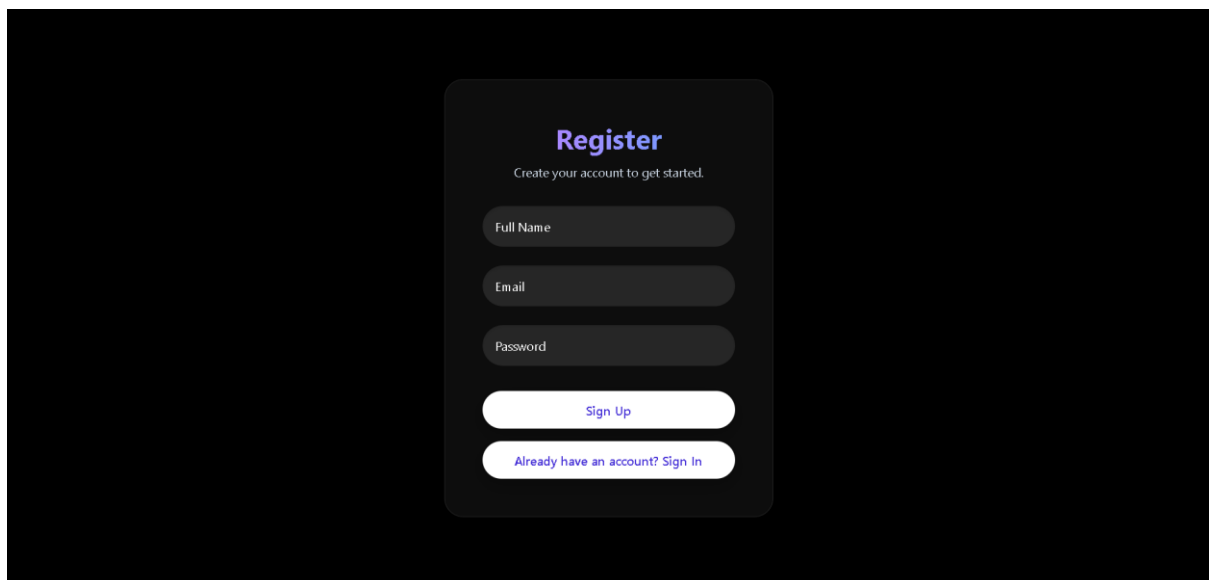


Figure 11: Login Page



The login page features a dark background with a central light gray rounded rectangle. At the top of this rectangle is the word "Login" in bold black text. Below it is a smaller line of text: "Welcome back! Please sign in to your account." There are two input fields: "Email" and "Password", both with light gray borders and rounded corners. Below these fields is a white rounded button with the text "Sign In" in black. At the bottom of the rectangle is another white rounded button with the text "Don't have an account? Register Now" in black.

Figure 12: Register Page



The register page features a dark background with a central light gray rounded rectangle. At the top of this rectangle is the word "Register" in bold black text. Below it is a smaller line of text: "Create your account to get started." There are three input fields: "Full Name", "Email", and "Password", all with light gray borders and rounded corners. Below these fields is a white rounded button with the text "Sign Up" in black. At the bottom of the rectangle is another white rounded button with the text "Already have an account? Sign In" in black.

Figure 13: Dashboard - Main Page

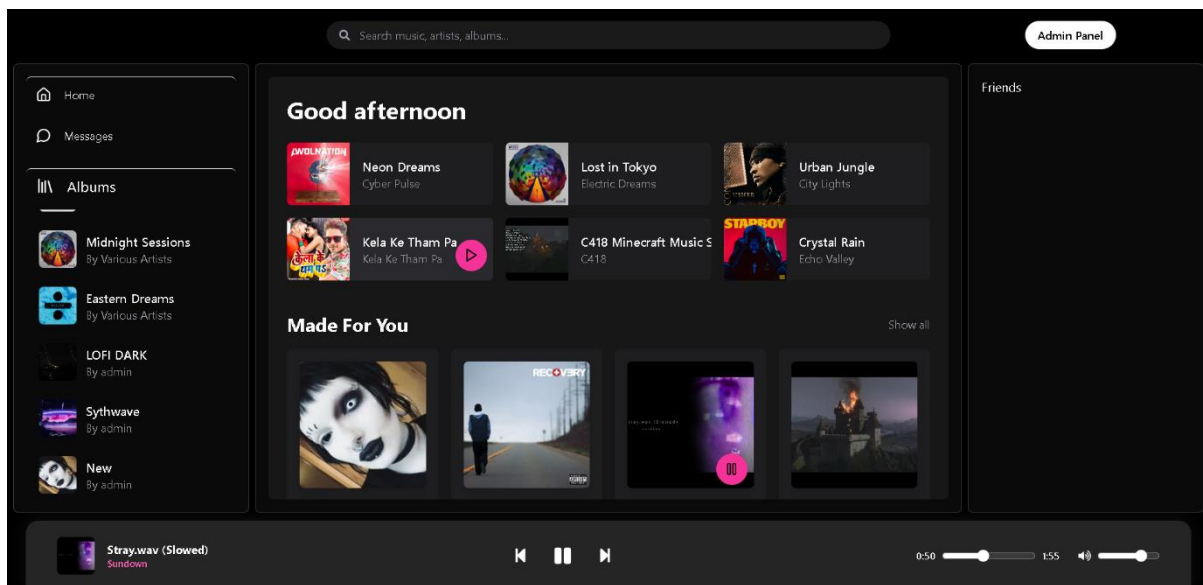


Figure 14: Dashboard - Trending

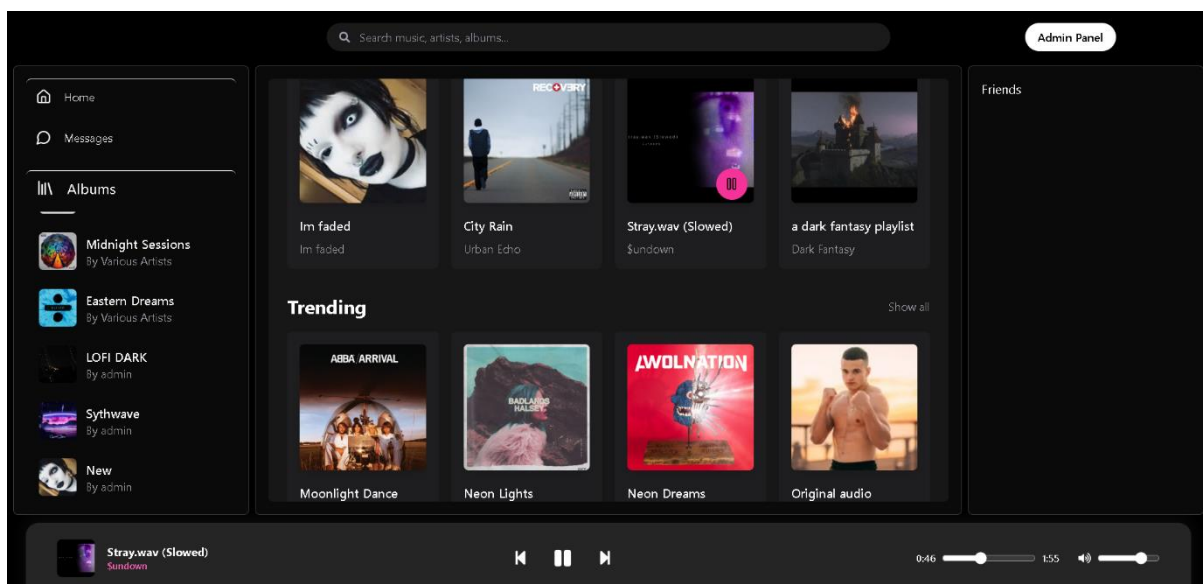


Figure 15: : Dashboard - Album Page (1)

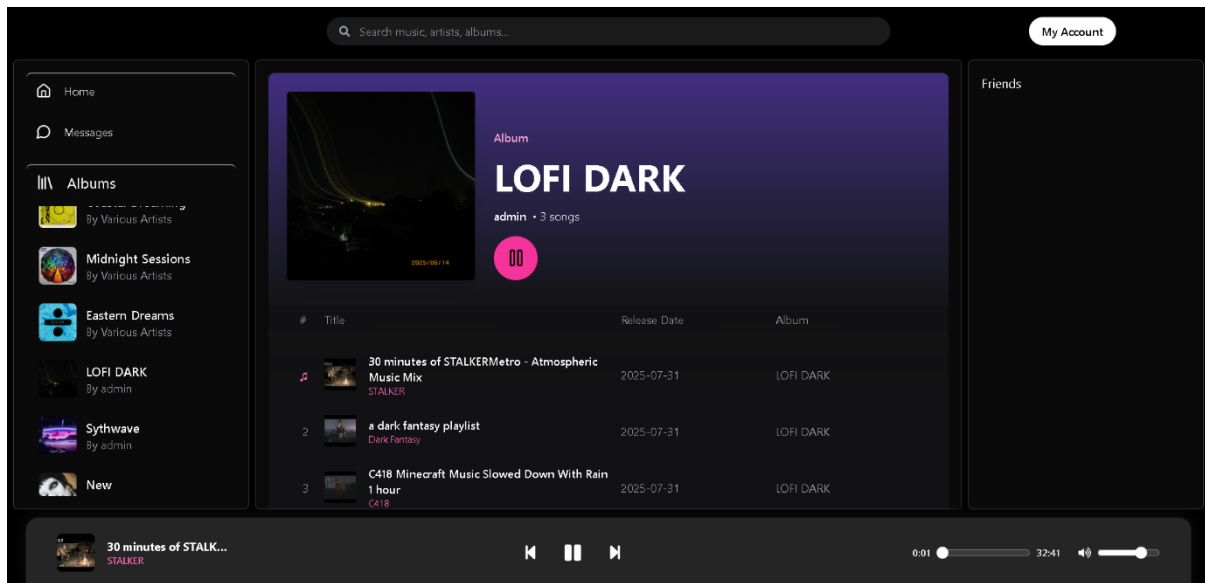


Figure 16: Dashboard - ALbum Page (2)

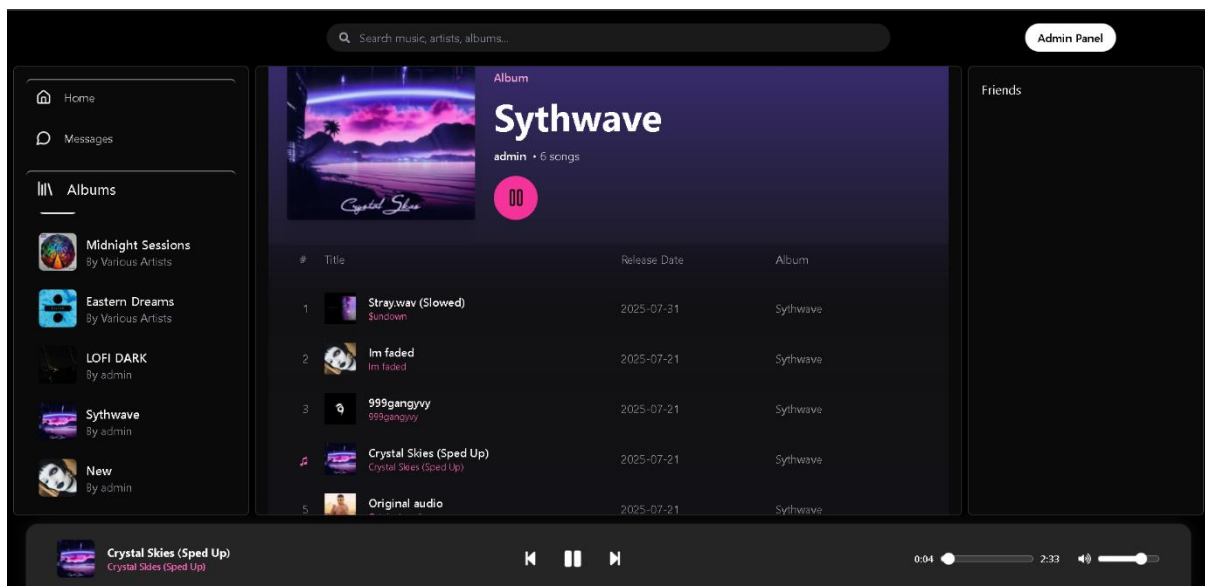


Figure 17: Dashboard - Search Suggestion

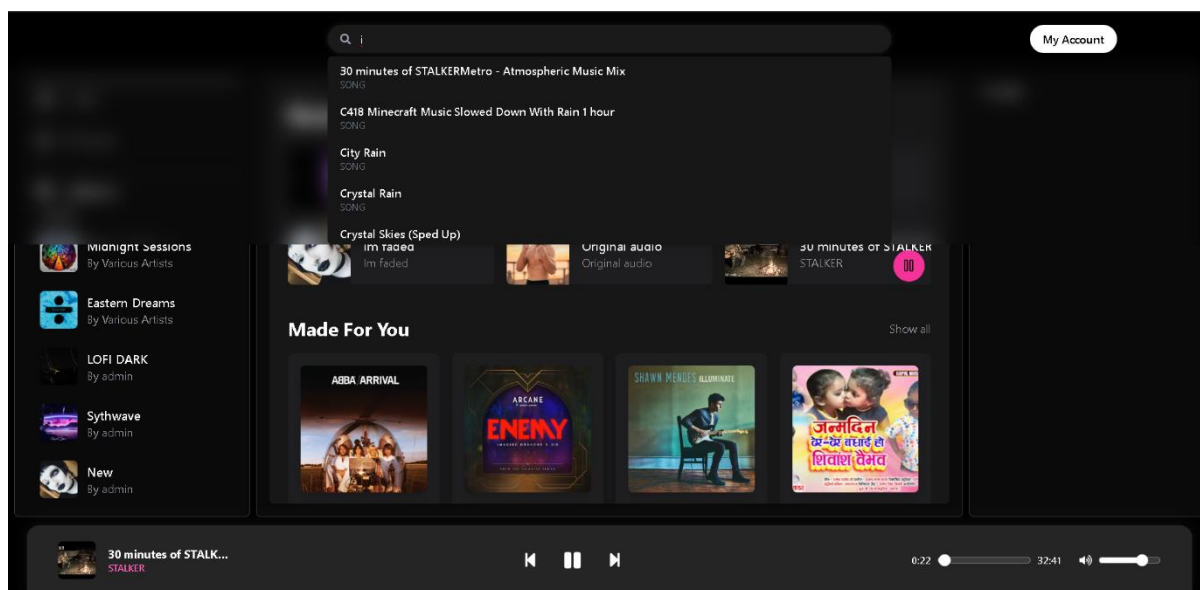


Figure 18: Admin Page - Songs Page

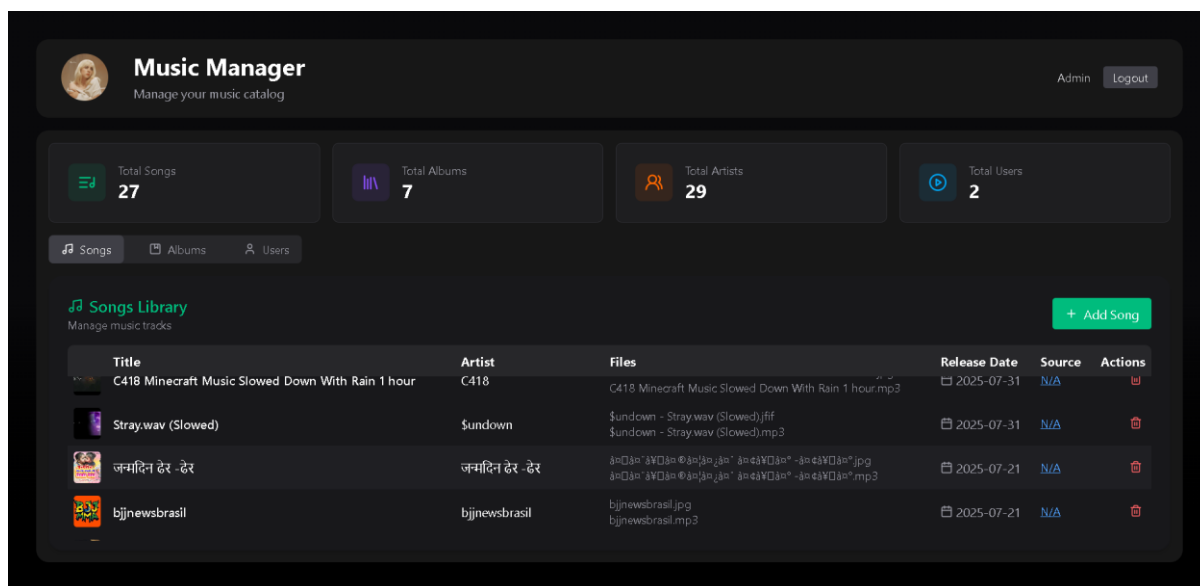


Figure 19: Admin Page - Add Song

The screenshot shows the 'Music Manager' Admin Page with a modal for adding a new song. The modal has the following fields and options:

- ☐ Convert from Instagram Reel URL
- Song Name:
- Artist Name:
- Album Name (optional):
- Audio File: Choose Audio File
- Image File: Choose Image File

Buttons: Cancel, Add Song

The background shows the 'Songs Library' with a table of songs:

Title	Release Date	Source	Actions
30 minutes of STALKERMetro - Atmospheric Music Mix	2025-07-31	N/A	
a dark fantasy playlist	2025-07-31	N/A	
C418 Minecraft Music Slowed Down With Rain 1 hour	2025-07-31	N/A	
Stray.wav (Slowed)	2025-07-31	N/A	

Figure 20: Admin Page - Album Page

The screenshot shows the 'Music Manager' Admin Page with the 'Albums Library' section. The table lists the following albums:

Album Name	Artist	Files	Release Date	Actions
Midnight Sessions	Various Artists	N/A	2025-07-21	
Eastern Dreams	Various Artists	N/A	2025-07-21	
LOFI DARK	admin	N/A	2025-07-31	
Synthwave	admin	N/A	2025-07-31	

Figure 21: Admin Page - Add Album

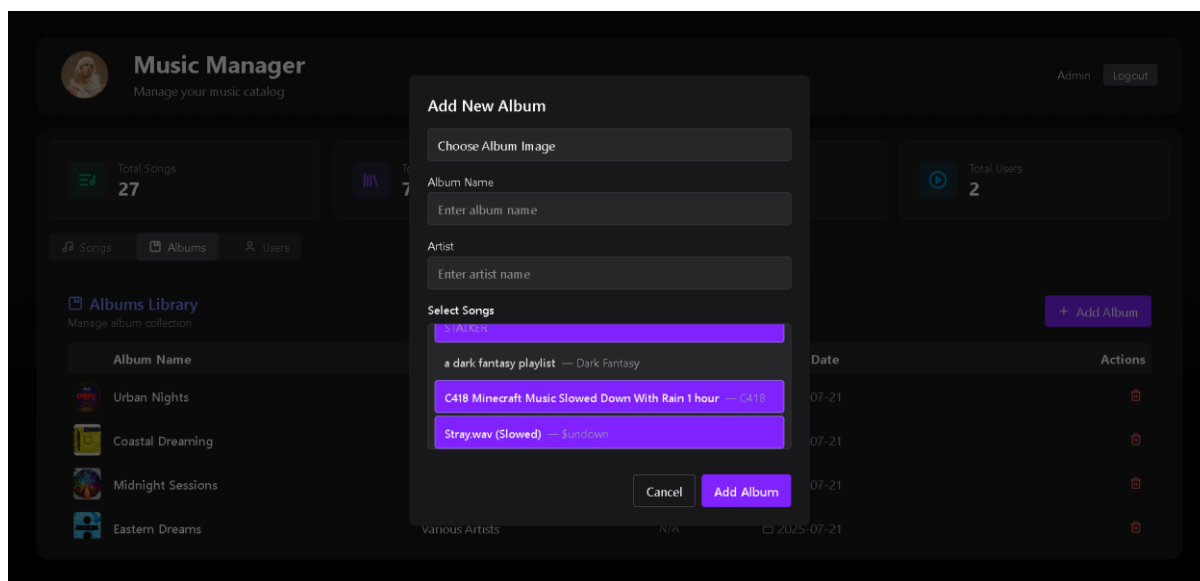
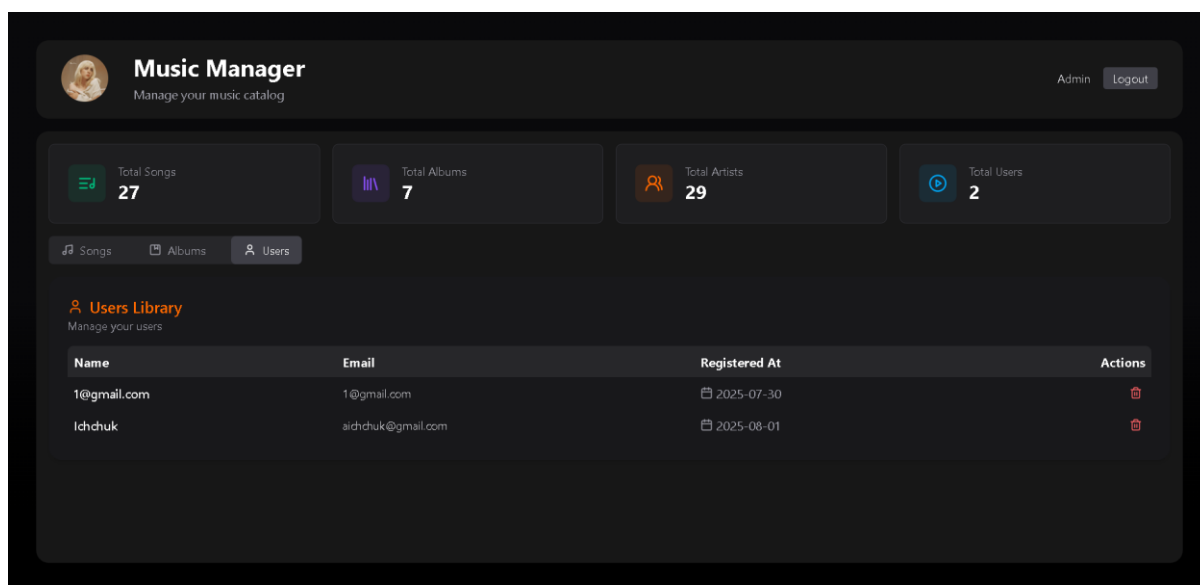


Figure 22: Admin Page - Users Page



4. Test Case Logs:

Figure 23: Frontend Tests

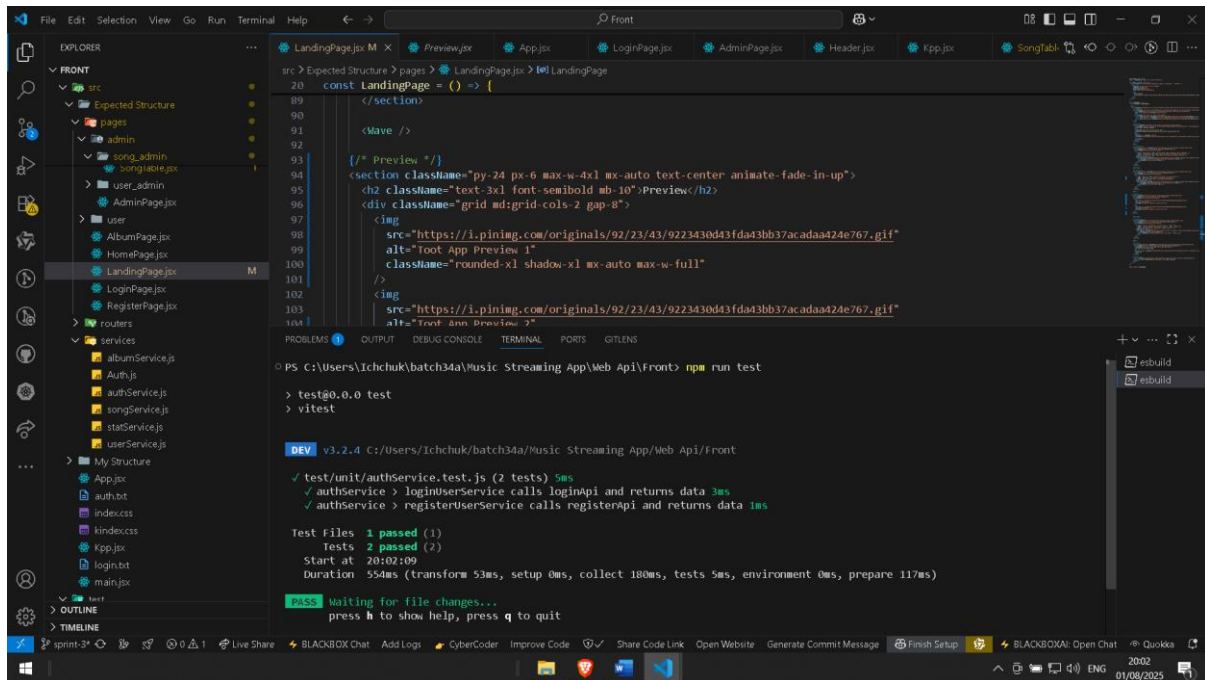


Figure 24: Backend Tests - 1

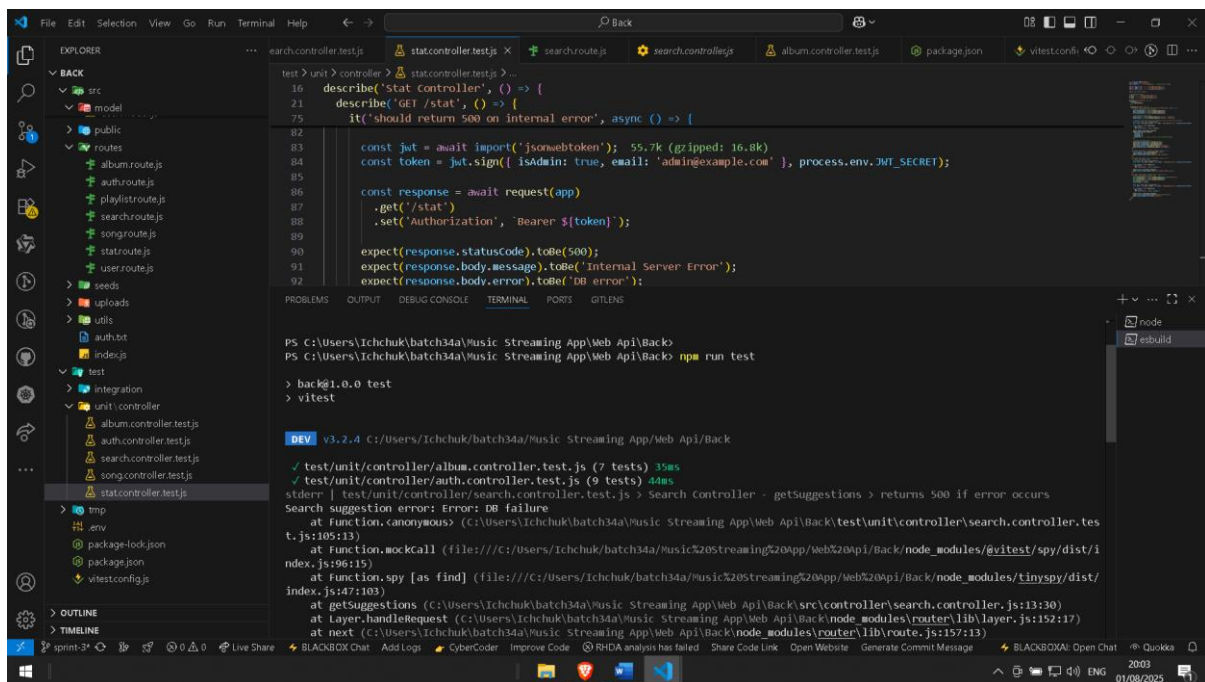


Figure 25: Backend Tests - 2

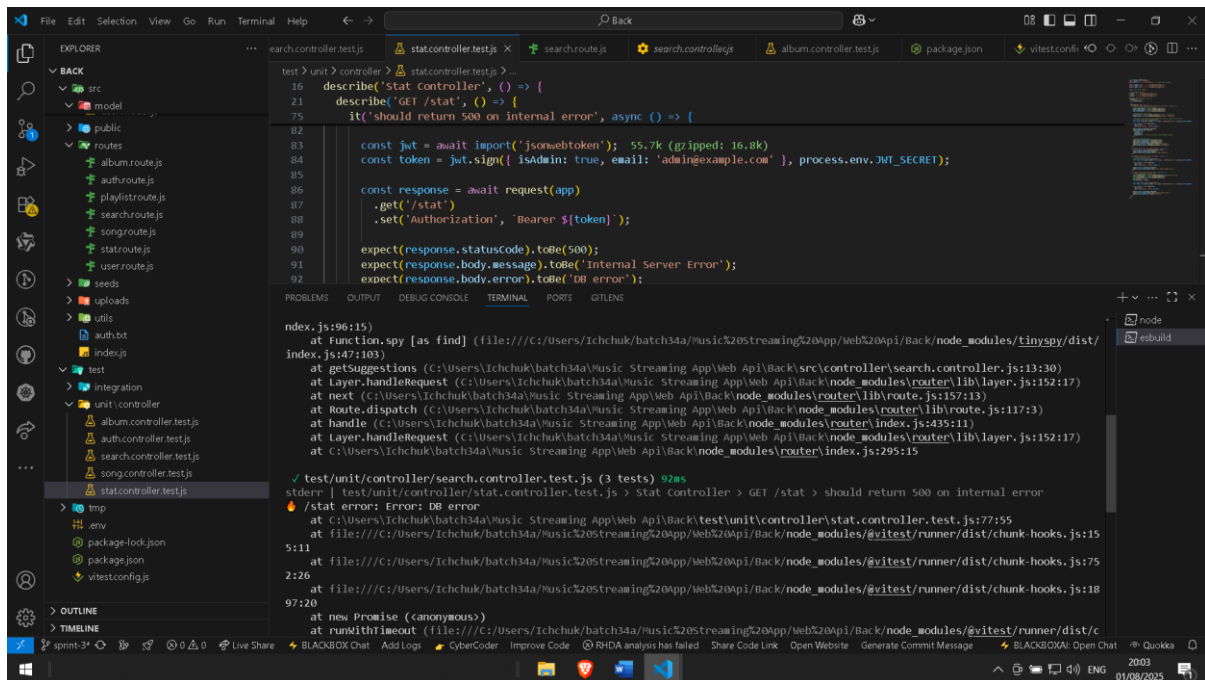


Figure 26: Backend Tests - 3

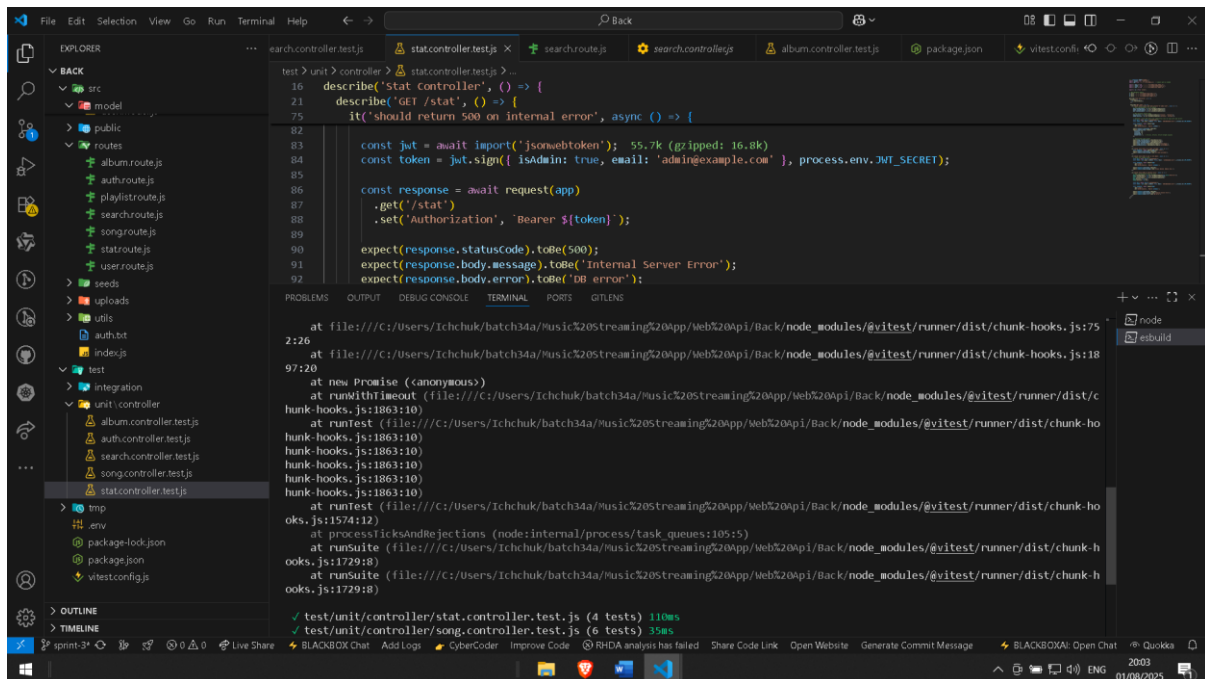
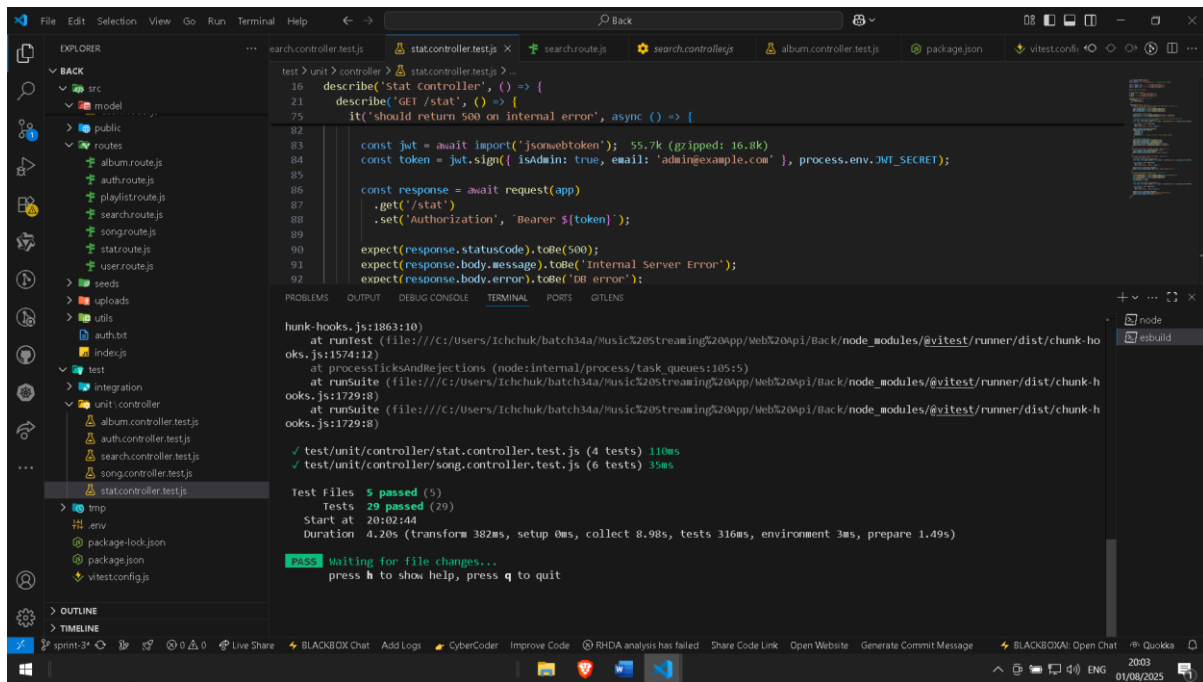


Figure 27: Backend Tests - 4



ER Diagram

Figure 28: ER Diagram

